

Filters and Histograms

This module assumes knowledge of the python programming language and familiarity with a python development environment. You should also be familiar with the concepts of OpenCV introduced in the previous practical.

We will be using **OpenCV (version 4.1.x)** computer vision library with **Python 3.7.x**. Whilst OpenCV is primarily aimed at computer vision tasks (i.e., image understanding and analysis), it is built on a central image processing and manipulation component that is both well-documented, open-source and free. It has already been setup for use on the Lab PCs (Windows and Linux) but you may alternatively use your own development environment on your own hardware. Instruction guides on installing OpenCV on your own hardware is provided on Ultra.

OpenCV has full documentation available at: <https://docs.opencv.org/4.1.1/>

Part 1: Applying Image Filters

1. Construct your own bespoke mask for image filtering and apply it on a test image using the `cv2.filter2D()` OpenCV function. You can use the file **peppers.png** (available on Ultra) or you might as well just cut the middleman and start implementing all the tasks on videos captured from a webcam or a loaded video (this set of instructions will eventually ask this of you anyway.)

This can be done by simply constructing the mask as a numpy array, as in the following example:

```
mask = np.ones((5,5),np.float32)/25
filtered_img = cv2.filter2D(img,-1,mask)
```

Here we created a 5×5 mask with each element having a value $1/25$, thus implementing the 5×5 mean filter.

Experiment with various masks of various sizes, including the Gaussian and Laplacian masks shown in lectures.

In some cases, you may need to scale (multiply) or add a constant to your output or simply normalise the image to make it visible.

2. Write a Laplacian filtering program using the `cv2.Laplacian()` OpenCV function.

The `cv2.Laplacian()` OpenCV function uses a more sophisticated discretisation of the Laplace operator than the one we derived in the lectures. It is based on a discretisation of first order derivatives with **Sobel** masks of various sizes. Read the documentation in the OpenCV manual:

https://docs.opencv.org/4.1.2/d5/db5/tutorial_laplace_operator.html

https://docs.opencv.org/4.1.2/d2/d2c/tutorial_sobel_derivatives.html

Try experimenting with Laplacian masks of different sizes.

You may once again need to scale or add a constant to your output (or simply normalise the image) to make it visible.

3. Based on your earlier use of the Laplacian filter in Task 1 and the outline of Laplacian based edge sharpening in Lecture 5, develop your own program to perform edge sharpening.

We saw a very crude demo during the lecture but try implementing your own (better) version.

Instead of the simple subtraction we discussed in Lecture 5, you might use a weighted operation:

$$I_{\text{out}} = I_{\text{in}} - (k * I_{\text{laplacian}})$$

for some value of k in $[0,1]$. k would be a tuneable parameter that can be controlled with a trackbar (we have seen examples of these in the lecture demos).

4. Develop your own program to perform **Laplacian of Gaussian (LoG)** filtering.

The naïve implementation of applying first a Gaussian filter on the image and then a Laplacian filter on the result is not the most efficient computationally.

For a more efficient implementation, notice that discrete convolution is essentially an operation between two matrices, that is, we can convolve a mask with another mask, and moreover this operation is associative. That means that:

$$L * (G * I) = (L * G) * I$$

where I is the image, G a Gaussian mask, L a Laplacian mask and $*$ denotes the convolution operation.

In other words, you should try to first convolve the Gaussian mask with the Laplacian mask (essentially precomputing the LoG filter) and then convolve the image with the result.

5. You knew it was coming. Implement all of the previous tasks on videos either captured from a live camera or the video included on Ultra (**video.avi**).

Part 2: Histograms

1. Download and Run the code '[histogram.py](#)' available on Ultra.

Extend this program to plot the histograms of each of the R, G and B colour channels from an incoming video.

Plot each in its corresponding colour and overlay them as a single image as per the RGB histogram examples in lectures (you can choose whether to use bar or line plots).

You can see an example of this should roughly look like on Ultra ('[histogram-peppers.png](#)')

Hints: you may find the Python OpenCV/NumPy notation `image[:, :, c]` useful. Here, `c` is the channel you wish to reference. For example, `B_channel = bgr_img[:, :, 0]`.

Note that OpenCV uses BGR colour channel ordering by default.

You may also find the OpenCV functions `cv2.split()` and `cv2.merge()` very useful.

2. Download the code "[equalise_histogram.py](#)" from the publicly available code demos on GitHub: https://github.com/atapour/ip-python-opencv/blob/main/equalise_histogram.py

Extend this program to perform histogram equalization on each of the R, G and B colour channels from an incoming image or video frame.

How does the result look? There might be better ways of doing this, which we will learn about later.

3. If you have time, implement your own version of `cv2.equalizeHist()` function from scratch.

There are a few useful tutorials to reinforce your understanding of histograms:

https://docs.opencv.org/4.1.1/d8/dbc/tutorial_histogram_calculation.html

https://docs.opencv.org/4.1.1/d4/d1b/tutorial_histogram_equalization.html

https://docs.opencv.org/4.1.1/d8/dc8/tutorial_histogram_comparison.html